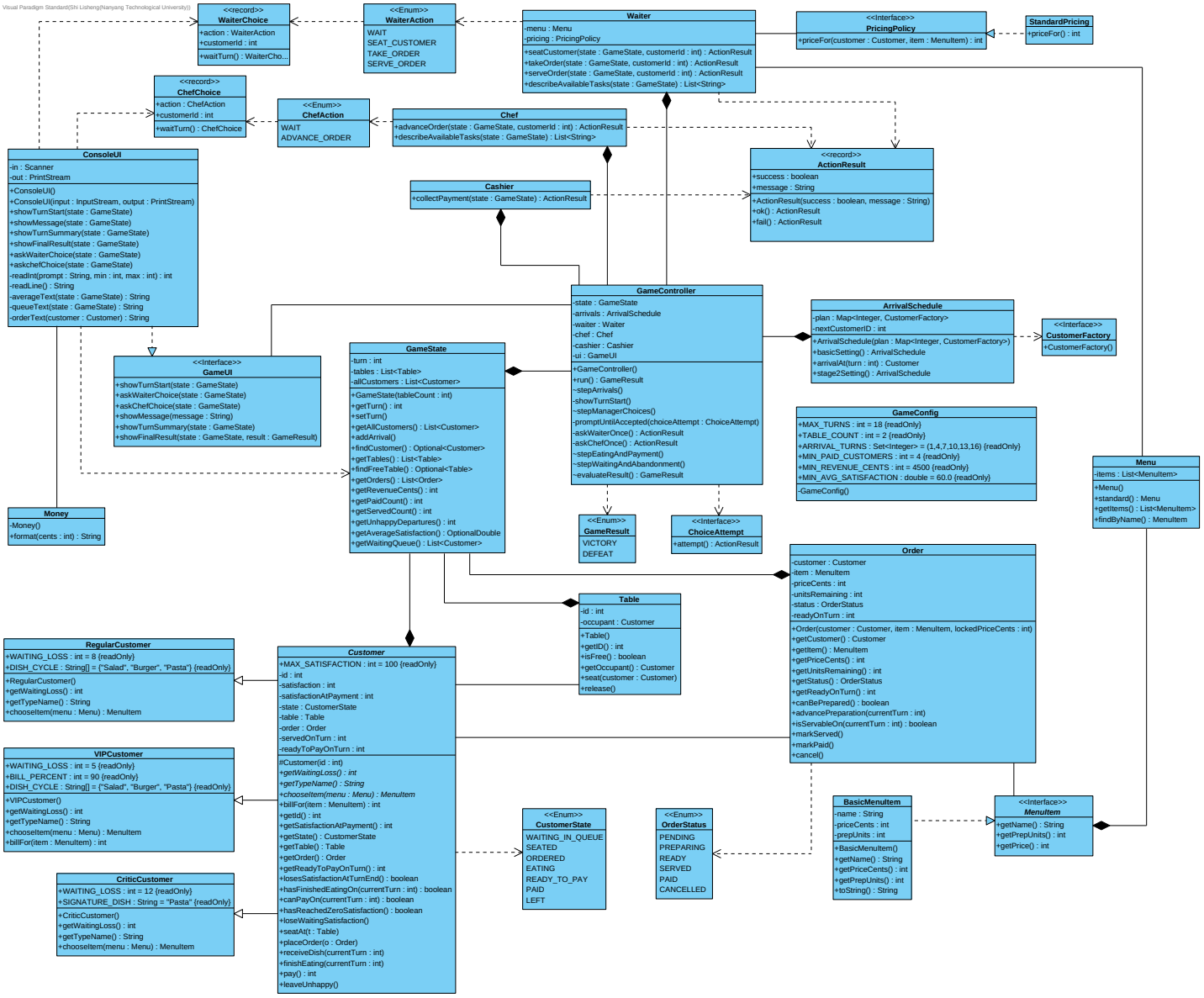
Visual Paradigm Standard(Shi Lisheng(Nanyang Technological University))
<<record>>
WaiterChoice
+action : WaiterAction
+customerId : int
+waitTurn() : WaiterCho...
<<Enum>>
WaiterAction
WAIT
SEAT_CUSTOMER
TAKE_ORDER
SERVE_ORDER
Waiter
-menu : Menu
-pricing : PricingPolicy
+seatCustomer(state : GameState, customerId : int) : ActionResult
+takeOrder(state : GameState, customerId : int) : ActionResult
+serveOrder(state : GameState, customerId : int) : ActionResult
+describeAvailableTasks(state : GameState) : List<String>
<<Interface>>
PricingPolicy
+priceFor(customer : Customer, item : MenuItem) : int
StandardPricing
+priceFor() : int
<<record>>
ChefChoice
+action : ChefAction
+customerId : int
+waitTurn() : ChefChoice
<<Enum>>
ChefAction
WAIT
ADVANCE_ORDER
Chef
+advanceOrder(state : GameState, customerId : int) : ActionResult
+describeAvailableTasks(state : GameState) : List<String>
ConsoleUI
-in : Scanner
-out : PrintStream
+ConsoleUI()
+ConsoleUI(input : InputStream, output : PrintStream)
+showTurnStart(state : GameState)
+showMessage(state : GameState)
+showTurnSummary(state : GameState)
+showFinalResult(state : GameState)
+askWaiterChoice(state : GameState)
+askchefChoice(state : GameState)
-readInt(prompt : String, min : int, max : int) : int
-readLine() : String
-averageText(state : GameState) : String
-queueText(state : GameState) : String
-orderText(customer : Customer) : String
Cashier
+collectPayment(state : GameState) : ActionResult
<<record>>
ActionResult
+success : boolean
+message : String
+ActionResult(success : boolean, message : String)
+ok() : ActionResult
+fail() : ActionResult
GameController
-state : GameState
-arrivals : ArrivalSchedule
-waiter : Waiter
-chef : Chef
-cashier : Cashier
-ui : GameUI
+GameController()
+run() : GameResult
~stepArrivals()
-showTurnStart()
~stepManagerChoices()
-promptUntilAccepted(choiceAttempt : ChoiceAttempt)
-askWaiterOnce() : ActionResult
-askChefOnce() : ActionResult
~stepEatingAndPayment()
~stepWaitingAndAbandonment()
-evaluateResult() : GameResult
ArrivalSchedule
-plan : Map<Integer, CustomerFactory>
-nextCustomerID : int
+ArrivalSchedule(plan : Map<Integer, CustomerFactory>)
+basicSetting() : ArrivalSchedule
+arrivalAt(turn : int) : Customer
+stage2Setting() : ArrivalSchedule
<<Interface>>
CustomerFactory
+CustomerFactory()
<<Interface>>
GameUI
+showTurnStart(state : GameState)
+askWaiterChoice(state : GameState)
+askChefChoice(state : GameState)
+showMessage(message : String)
+showTurnSummary(state : GameState)
+showFinalResult(state : GameState, result : GameResult)
GameState
-turn : int
-tables : List<Table>
-allCustomers : List<Customer>
+GameState(tableCount : int)
+getTurn() : int
+setTurn()
+getAllCustomers() : List<Customer>
+addArrival()
+findCustomer() : Optional<Customer>
+getTables() : List<Table>
+findFreeTable() : Optional<Table>
+getOrders() : List<Order>
+getRevenueCents() : int
+getPaidCount() : int
+getServedCount() : int
+getUnhappyDepartures() : int
+getAverageSatisfaction() : OptionalDouble
+getWaitingQueue() : List<Customer>
GameConfig
+MAX_TURNS : int = 18 {readOnly}
+TABLE_COUNT : int = 2 {readOnly}
+ARRIVAL_TURNS : Set<Integer> = (1,4,7,10,13,16) {readOnly}
+MIN_PAID_CUSTOMERS : int = 4 {readOnly}
+MIN_REVENUE_CENTS : int = 4500 {readOnly}
+MIN_AVG_SATISFACTION : double = 60.0 {readOnly}
-GameConfig()
Menu
-items : List<MenuItem>
+Menu()
+standard() : Menu
+getItems() : List<MenuItem>
+findByName() : MenuItem
Money
-Money()
+format(cents : int) : String
<<Enum>>
GameResult
VICTORY
DEFEAT
<<Interface>>
ChoiceAttempt
+attempt() : ActionResult
Order
-customer : Customer
-item : MenuItem
-priceCents : int
-unitsRemaining : int
-status : OrderStatus
-readyOnTurn : int
+Order(customer : Customer, item : MenuItem, lockedPriceCents : int)
+getCustomer() : Customer
+getItem() : MenuItem
+getPriceCents() : int
+getUnitsRemaining() : int
+getStatus() : OrderStatus
+getReadyOnTurn() : int
+canBePrepared() : boolean
+advancePreparation(currentTurn : int)
+isServableOn(currentTurn : int) : boolean
+markServed()
+markPaid()
+cancel()
Table
-id : int
-occupant : Customer
+Table()
+getID() : int
+isFree() : boolean
+getOccupant() : Customer
+seat(customer : Customer)
+release()
RegularCustomer
+WAITING_LOSS : int = 8 {readOnly}
+DISH_CYCLE : String[] = {"Salad", "Burger", "Pasta"} {readOnly}
+RegularCustomer()
+getWaitingLoss() : int
+getTypeName() : String
+chooseItem(menu : Menu) : MenuItem
Customer
+MAX_SATISFACTION : int = 100 {readOnly}
-id : int
-satisfaction : int
-satisfactionAtPayment : int
-state : CustomerState
-table : Table
-order : Order
-servedOnTurn : int
-readyToPayOnTurn : int
#Customer(id : int)
+getWaitingLoss() : int
+getTypeName() : String
+chooseItem(menu : Menu) : MenuItem
+billFor(item : MenuItem) : int
+getId() : int
+getSatisfactionAtPayment() : int
+getState() : CustomerState
+getTable() : Table
+getOrder() : Order
+getReadyToPayOnTurn() : int
+losesSatisfactionAtTurnEnd() : boolean
+hasFinishedEatingOn(currentTurn : int) : boolean
+canPayOn(currentTurn : int) : boolean
+hasReachedZeroSatisfaction() : boolean
+loseWaitingSatisfaction()
+seatAt(t : Table)
+placeOrder(o : Order)
+receiveDish(currentTurn : int)
+finishEating(currentTurn : int)
+pay() : int
+leaveUnhappy()
VIPCustomer
+WAITING_LOSS : int = 5 {readOnly}
+BILL_PERCENT : int = 90 {readOnly}
+DISH_CYCLE : String[] = {"Salad", "Burger", "Pasta"} {readOnly}
+VIPCustomer()
+getWaitingLoss() : int
+getTypeName() : String
+chooseItem(menu : Menu) : MenuItem
+billFor(item : MenuItem) : int
CriticCustomer
+WAITING_LOSS : int = 12 {readOnly}
+SIGNATURE_DISH : String = "Pasta" {readOnly}
+CriticCustomer()
+getWaitingLoss() : int
+getTypeName() : String
+chooseItem(menu : Menu) : MenuItem
<<Enum>>
CustomerState
WAITING_IN_QUEUE
SEATED
ORDERED
EATING
READY_TO_PAY
PAID
LEFT
<<Enum>>
OrderStatus
PENDING
PREPARING
READY
SERVED
PAID
CANCELLED
BasicMenuItem
-name : String
-priceCents : int
-prepUnits : int
+BasicMenuItem()
+getName() : String
+getPriceCents() : int
+getPrepUnits() : int
+toString() : String
<<Interface>>
MenuItem
+getName() : String
+getPrepUnits() : int
+getPrice() : int